



Matematicko-fyzikální fakulta

Univerzita Karlova

Stanislav Dochynets

Informatický ústav Univerzity Karlovy

Rozhraní pro plantri **a** geng

9. září 2026

- Generování grafů se používá při hledání protipříkladů, ověřování hypotéz a testování grafových algoritmů.
- Existují velmi efektivní generátory:
 - `geng` pro obecné neorientované grafy,
 - `plantri` pro rovinné grafy.
- Jejich přímé použití z vlastního C++ programu je méně pohodlné:
 - nízkoúrovňové C rozhraní,
 - callbacky přes makra nebo pluginy,
 - nutnost znát interní reprezentaci grafu.

- Navrhnout a implementovat knihovnu **PlanG**.
- Zpřístupnit `geng` a `plantri` z moderního C++ kódu.
- Poskytnout:
 - jednotné rozhraní pro konfiguraci a spuštění generování,
 - callbacky pro filtrování, prořezávání a zpracování výsledků,
 - pohled na graf bez automatického kopírování,
 - částečnou kompatibilitu s Boost Graph Library.
- Rozšířit backend `geng` o barevné třídy.

- Základním vstupním bodem je šablona:

`Generator<Backend>`

- Backend určuje konkrétní původní generátor:
 - `Generator<geng::Backend>`,
 - `Generator<plantri::Backend>`.
- Společná vrstva poskytuje stejný způsob spuštění a registrace callbacků.
- Specifické volby zůstávají v příslušném backendu.

- `setFilter`
 - filtruje dokončené grafy,
 - rozhoduje, zda graf ponechat nebo odmítnout.
- `setPreprune`
 - pracuje už s částečně vytvořenými grafy,
 - může předčasně zahodit celou větev generování.
- `setOutproc`
 - umožňuje vlastní zpracování výstupních grafů.
- Callbacky jsou běžné C++ funkce nebo lambda výrazy.

- Vygenerovaný graf se automaticky nekopíruje do nové datové struktury.
- Callback dostane objekt `GraphView`:
 - u `geng` pohled na bitovou reprezentaci nauty,
 - u `plantri` pohled na pole stupňů a orientované hrany.
- Při vytvoření `GraphView` se kopíruje jen několik ukazatelů a základních údajů o grafu.
- Nad `GraphView` lze používat vybrané funkce ve stylu Boost Graph Library.

- Objekty `GraphView` jsou přizpůsobeny vybraným konceptům Boost Graph Library.
- Pro oba backendy jsou definovány:
 - typy vrcholů a hran,
 - iterátory přes vrcholy, hrany a sousedy,
 - globální funkce jako `vertices(g)`, `edges(g)`, `source(e,g)` a `target(e,g)`.
- Uživatel může v callbacku spustit vybrané obecné grafové algoritmy bez převodu grafu do jiné datové struktury.
- Implementace respektuje rozdílné reprezentace:
 - bitové množiny v `geng`,
 - orientované hrany v `plantri`.

- Původní generátory zůstávají nejnižší vrstvou.
- Nad nimi je tenká C mezivrstva:
 - zpřístupňuje vstupní funkce a vybraný globální stav,
 - napojuje původní callback mechanismy.
- C++ bridge:
 - uchovává uživatelské callbacky,
 - vytváří `GraphView`,
 - zabraňuje průchodu C++ výjimek přes C kód.
- Nejvyšší vrstva: backendy a `Generator<Backend>`.

- Knihovna zachovává původní algoritmy generátorů (výjimkou je rozšíření `geng` o podporu barevných tříd).
- `geng` a `plantri` nejsou spouštěny jako externí programy, ale jako součást knihovny.
- Konfigurace z C++ objektu se převede na stejné argumenty, jaké by uživatel zadal na příkazové řádce.
- Původní callbacky generátorů jsou uvnitř knihovny napojeny na C++ funkce zadané uživatelem.
- Uživatel tak místo maker a pluginů píše běžný C++ kód.

- geng byl rozšířen o barevné třídy
- Při přidávání nového vrcholu se kromě množiny sousedů zkouší také jeho barva.
- Generátor průběžně kontroluje:
 - dolní a horní meze velikostí barevných tříd,
 - zda lze omezení ještě splnit se zbývajícími vrcholy.
- Kanonické testy respektují barvy pomocí partition rozdělené podle barevných tříd.
- Zakotvené vrcholy jsou reprezentovány jako jednoprvkové barevné třídy.

Generování obarvených grafů(pseudokod)

```
extend(graf G na n vrcholech):  
  pro každou přípustnou množinu sousedů nového vrcholu:  
    pokud obarvení není aktivní:  
      pokračuj původním algoritmem geng  
  
    pokud obarvení je aktivní:  
      pro každou barvu c:  
        přiřad' novému vrcholu barvu c  
        zvyš počet vrcholů v barvě c  
  
    pokud lze ještě splnit meze barevných tříd:  
      proved' kanonický test s partition podle barev  
      pokud test projde:  
        pokračuj rekurzivně  
  
  vrat' přiřazení barvy zpět
```

- Funkční testování:
 - porovnání výstupu PlanG s nativními generátory,
 - testy callbacků,
 - manuální ověření barevných konfigurací backendu geng.
- Výkonové měření:
 - čisté generování bez výstupu,
 - režie prázdných callbacků,
 - srovnání callbackové implementace s pluginem `maxdeg.c`.
- Měřeno proti nativním programům geng a plantri.

- Při čistém generování byl výkon PlanG srovnatelný s nativními programy.
- Prázdné callbacky nepřidávaly významnou režii.
- Callbacková implementace omezení maximálního stupně v `plantri` byla pouze mírně pomalejší než nativní plugin.
- Barevné generování v `geng` je výrazně náročnější, protože řeší obecnější úlohu.

- Navržena a implementována knihovna PlanG.
- Jednotné C++ rozhraní pro `geng` a `plantri` (s možností rozšíření o další backendy).
- Callbacky pro filtrování, prořezávání a zpracování grafů.
- Integrace s vybranými koncepty Boost Graph Library.
- Rozšíření `geng` o barevné třídy a zakotvené vrcholy.
- Ověření funkčnosti a výkonové srovnání s původními generátory.

Děkuji za pozornost!